Generate binary dataset

Generate a synthetic binary classification dataset with two features and one class.

Here is a minimal example in Python using `sklearn.datasets.make_classification`:

```python
from sklearn.datasets import make_classification
import pandas as pd

# generate dataset
X, y = make_classification(
    n_samples=200,     # number of samples
    n_features=2,      # two features
    n_informative=2,   # both features are informative
    n_redundant=0,     # no redundant features
    n_clusters_per_class=1,
    random_state=42
)

# put into DataFrame
df = pd.DataFrame(X, columns=["feature1", "feature2"])
df["class"] = y

print(df.head())
```

This produces a binary classification dataset with two features (`feature1`, `feature2`) and one binary target column (`class`).

Do you want me to also plot it so you can visualize the class separation?

2/2

Write Jupyter notebook code to train logistic regression on this dataset twice: once using binary cross-entropy (log loss) and once using mean squared error. show accuracy and decision boundary plots for both.

Here is a complete Jupyter notebook script that trains logistic regression twice: once with **binary cross-entropy (log loss)** and once with **mean squared error (MSE)**. It also shows accuracy and decision boundaries for both.

python

```python
# %%
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, log_loss, mean_squared_error

# Generate synthetic dataset
X, y = make_classification(
    n_samples=200,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    random_state=42
)

# %%
# --- Model 1: Logistic Regression with Log Loss (default) ---
log_reg_ce = LogisticRegression(solver="lbfgs")
log_reg_ce.fit(X, y)
y_pred_ce = log_reg_ce.predict(X)
y_proba_ce = log_reg_ce.predict_proba(X)[:, 1]

acc_ce = accuracy_score(y, y_pred_ce)
loss_ce = log_loss(y, y_proba_ce)

print("Cross-Entropy Logistic Regression")
print("Accuracy:", acc_ce)
print("Log Loss:", loss_ce)

# %%
# --- Model 2: Logistic Regression with MSE (custom) ---
from sklearn.base import BaseEstimator, ClassifierMixin
from scipy.optimize import minimize
```

```
class LogisticRegressionMSE(BaseEstimator, ClassifierMixin):
    def __init__(self):
        self.w = None

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        Xb = np.hstack([np.ones((X.shape[0], 1)), X])  # bias term

        def mse_loss(w):
            preds = self.sigmoid(Xb @ w)
            return mean_squared_error(y, preds)

        res = minimize(mse_loss, np.zeros(Xb.shape[1]))
        self.w = res.x
        return self

    def predict_proba(self, X):
        Xb = np.hstack([np.ones((X.shape[0], 1)), X])
        probs = self.sigmoid(Xb @ self.w)
        return np.vstack([1 - probs, probs]).T

    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

# Train custom MSE logistic regression
log_reg_mse = LogisticRegressionMSE()
log_reg_mse.fit(X, y)
y_pred_mse = log_reg_mse.predict(X)
y_proba_mse = log_reg_mse.predict_proba(X)[:, 1]

acc_mse = accuracy_score(y, y_pred_mse)
loss_mse = mean_squared_error(y, y_proba_mse)

print("\nMSE Logistic Regression")
print("Accuracy:", acc_mse)
print("MSE Loss:", loss_mse)

# %%
# --- Decision Boundary Plot Function ---
def plot_decision_boundary(model, X, y, title):
```

```
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                         np.linspace(y_min, y_max, 200))
    grid = np.c_[xx.ravel(), yy.ravel()]

    Z = model.predict(grid)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k", cmap=plt.cm.coolwarm)
    plt.title(title)
    plt.show()

# Plot decision boundaries
plot_decision_boundary(log_reg_ce, X, y, f"Cross-Entropy Logistic Regression (Acc={acc_ce:.2f})")
plot_decision_boundary(log_reg_mse, X, y, f"MSE Logistic Regression (Acc={acc_mse:.2f})")
```

This notebook does the following:

- Generates a 2D binary classification dataset.
- Trains logistic regression with **cross-entropy** (default `LogisticRegression`).
- Trains logistic regression with **MSE** using a custom optimizer.
- Prints accuracy and loss values.
- Plots the decision boundaries for both models.

Do you want me to also add a side-by-side plot so you can visually compare the boundaries in one figure instead of two?

Explain the theoretical reason why binary cross-entropy is better than MSE for logistic regression.

Binary cross-entropy (log loss) is better for logistic regression because it matches the probabilistic model being used.

- **Logistic regression assumption:**
  The model assumes that the conditional distribution of the label $y \in \{0, 1\}$ given features x follows a Bernoulli distribution with parameter $p(x; \theta)$.

$$P(y \mid x; \theta) = p(x; \theta)^y \, (1 - p(x; \theta))^{1-y}, \quad p(x; \theta) = \sigma(\theta^T x)$$

The natural loss is the **negative log-likelihood**, which is binary cross-entropy:

$$L_{BCE} = -(y \log p(x;\theta) + (1-y)\log(1-p(x;\theta)))$$

- **Why BCE works better than MSE:**
  1. **Statistical consistency:** BCE is the maximum likelihood estimator for logistic regression, so it is statistically consistent and efficient. MSE is not tied to the Bernoulli likelihood.
  2. **Gradient behavior:** BCE has stronger gradients when predictions are very wrong (e.g., predicting $p = 0.01$ when $y = 1$). MSE gradients shrink in that case, slowing convergence.
  3. **Decision boundary:** Logistic regression with BCE directly optimizes class separation. With MSE, optimization tries to match real values 0 and 1, which distorts the probability estimation and can bias the decision boundary.
  4. **Calibration:** BCE ensures predicted probabilities are well-calibrated. MSE tends to push predictions toward the middle (0.5), leading to under-confident outputs.

In practice: BCE converges faster, yields better boundaries, and produces meaningful probabilities. MSE may still classify correctly but often with worse separation and poorer probability estimates.

Do you want me to also derive the gradient differences between BCE and MSE for logistic regression to make the contrast precise?

the accuracy for the BCI is 0.84 while for MSE is 0.835, how do you comment on this?

An accuracy difference of **0.84 vs. 0.835** is negligible.

Key points:

- **Small dataset:** With 200 samples, accuracy can fluctuate from random variation.
- **Same decision threshold (0.5):** Both BCE and MSE can end up with very similar boundaries, so classification accuracy may be close.
- **Accuracy is coarse:** It only counts correct vs. incorrect labels. It ignores probability quality, calibration, and learning dynamics.

The theoretical advantage of BCE appears more in:

- Faster convergence during training.
- Better probability estimates (important for downstream tasks, not just classification).
- More robustness in imbalanced or harder datasets.

So: **MSE can yield similar accuracy on simple or well-separated data, but BCE remains the principled choice for logistic regression.**

Do you want me to show a metric like **log loss or calibration curves** to highlight the difference more clearly than accuracy does?